# AUTOMATED MACHINE LEARNING (AUTOML) PLATFORM WITH DYNAMIC PIPELINE ROUTING

**Presented By**

**Jamil Aktar**
**Jayed Islam**
**Dilruba Khatun**

**Under the Guidance of**
**Kapil Dev Das**

**Department of Computer Science and Engineering**
**Domkal Institute of Engineering and Technology**

# The Data Science Bottleneck

## Dirty Data


Real-world tabular data is inherently messy. Missing numerical cells and unencoded strings cause standard ML algorithms to instantly fail.

## Manual Workflows

```
data = pd.read_csv('data.csv')
data.fillna(data.mean(), inplace=True) # Manual imputation

encoder = OneHotEncoder()
X = encoder.fit_transform(data[['category']]) # Manual encoding

model = RandomForestClassifier(n_estimators=100) # Manual tuning
model.fit(X, y)
```

Data scientists spend the majority of their time writing repetitive code for imputation, categorical encoding, and algorithm tuning, creating a massive barrier to entry.

# The Problem Statement

### Dirty Real-World Data

Real tabular data contains missing cells, varying scales, and categorical strings. Standard ML algorithms immediately crash when fed this raw data, requiring tedious manual cleaning.

### Vendor Lock-in & Costs

Commercial enterprise tools (like Google Vertex AI) are highly expensive and force users into proprietary cloud ecosystems, making local deployment difficult.

### UI Blocking Constraints

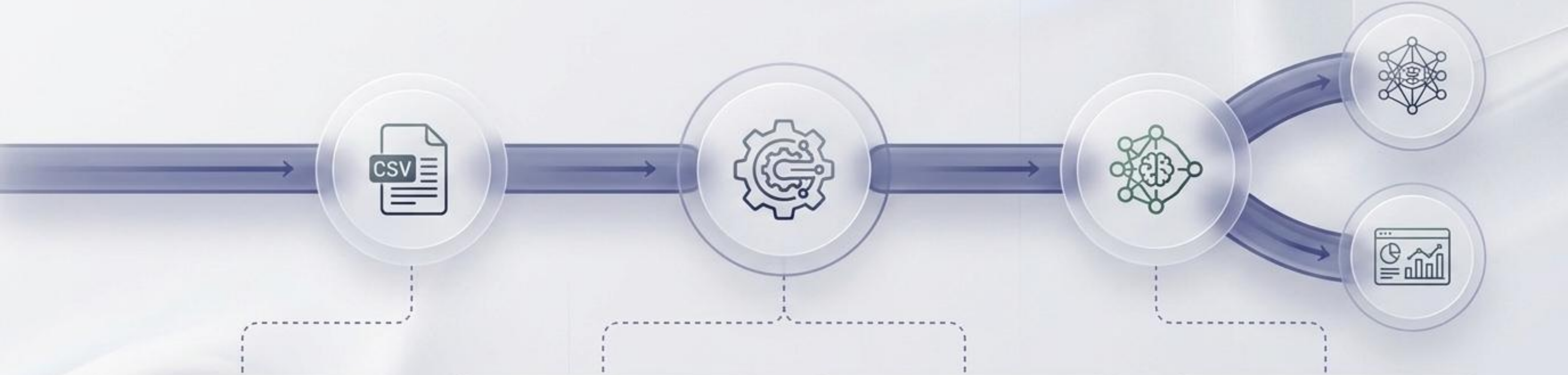
Open-source tools (like Auto-Sklearn) lack a GUI. When deployed on standard web servers, heavy ML training blocks the main thread, causing browsers to freeze and timeout.

# The AutoML Landscape

A diagnostic comparison of current market paradigms

| Platform Paradigm | Cost | User Interface | Primary Drawback / Advantage |
|---|---|---|---|
| Enterprise Cloud (e.g., Google Vertex AI) | Prohibitive | Full GUI | **Severe Vendor Lock-in** |
| Open-Source Libraries (e.g., Auto-Sklearn) | Free | None (Requires Python) | **Single-Threaded** (Blocks web servers) |
| **Our Proposed Platform** | Free / Open-Architecture | Next.js Drag-and-Drop Dashboard | **Fully Asynchronous, Decoupled Architecture** |

# The Paradigm Shift: A No-Code AutoML SaaS




**Automated Preprocessing**

Autonomous imputation and encoding

**Intelligent Routing**

Dynamic algorithm selection

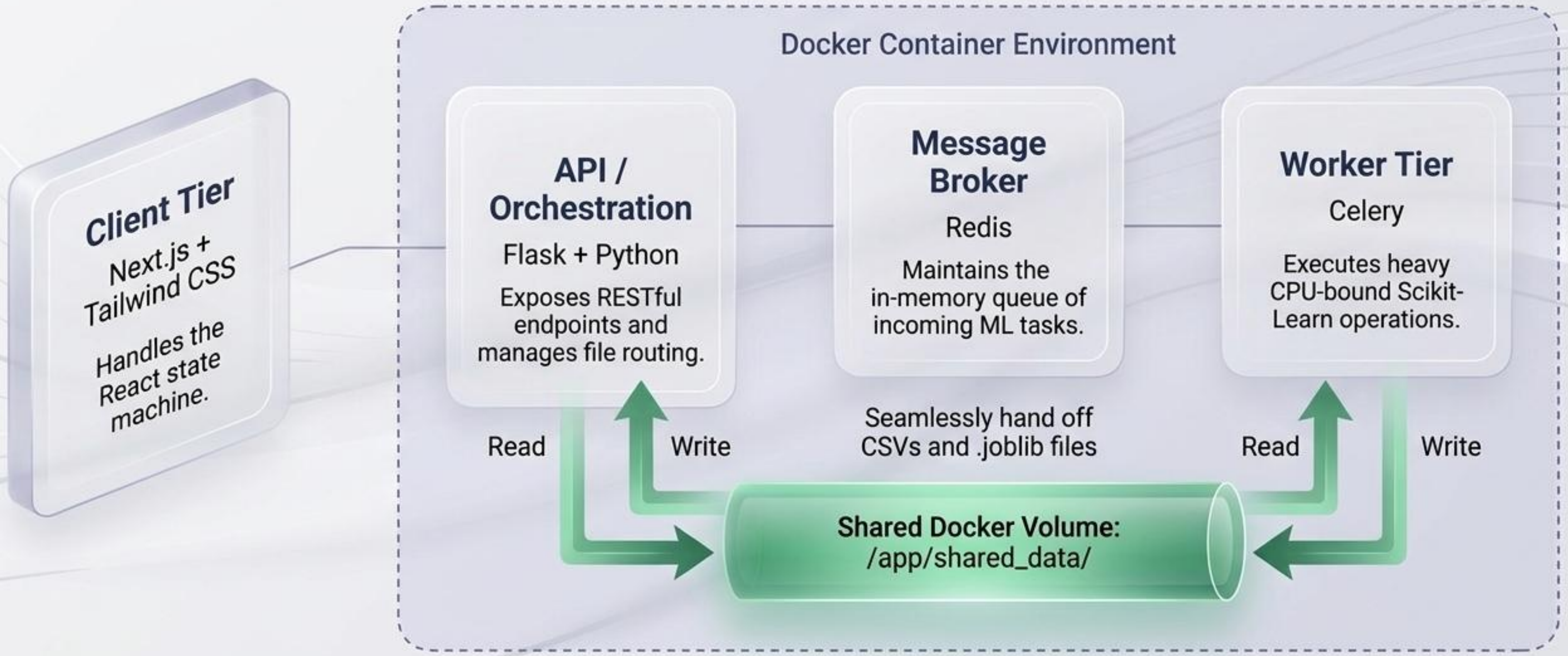
**Concurrent EDA**

Real-time statistical profiling

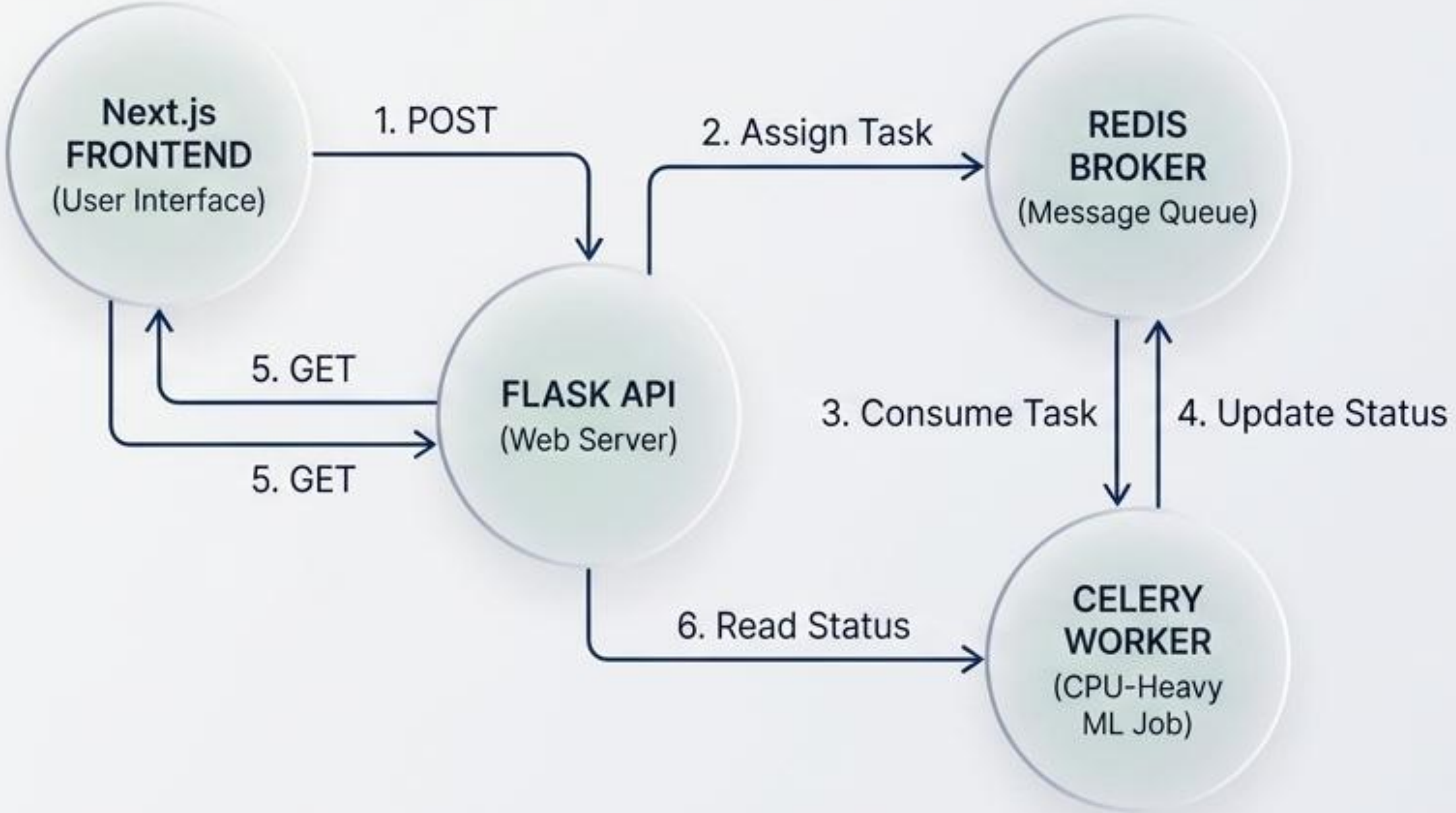
**Asynchronous UI**

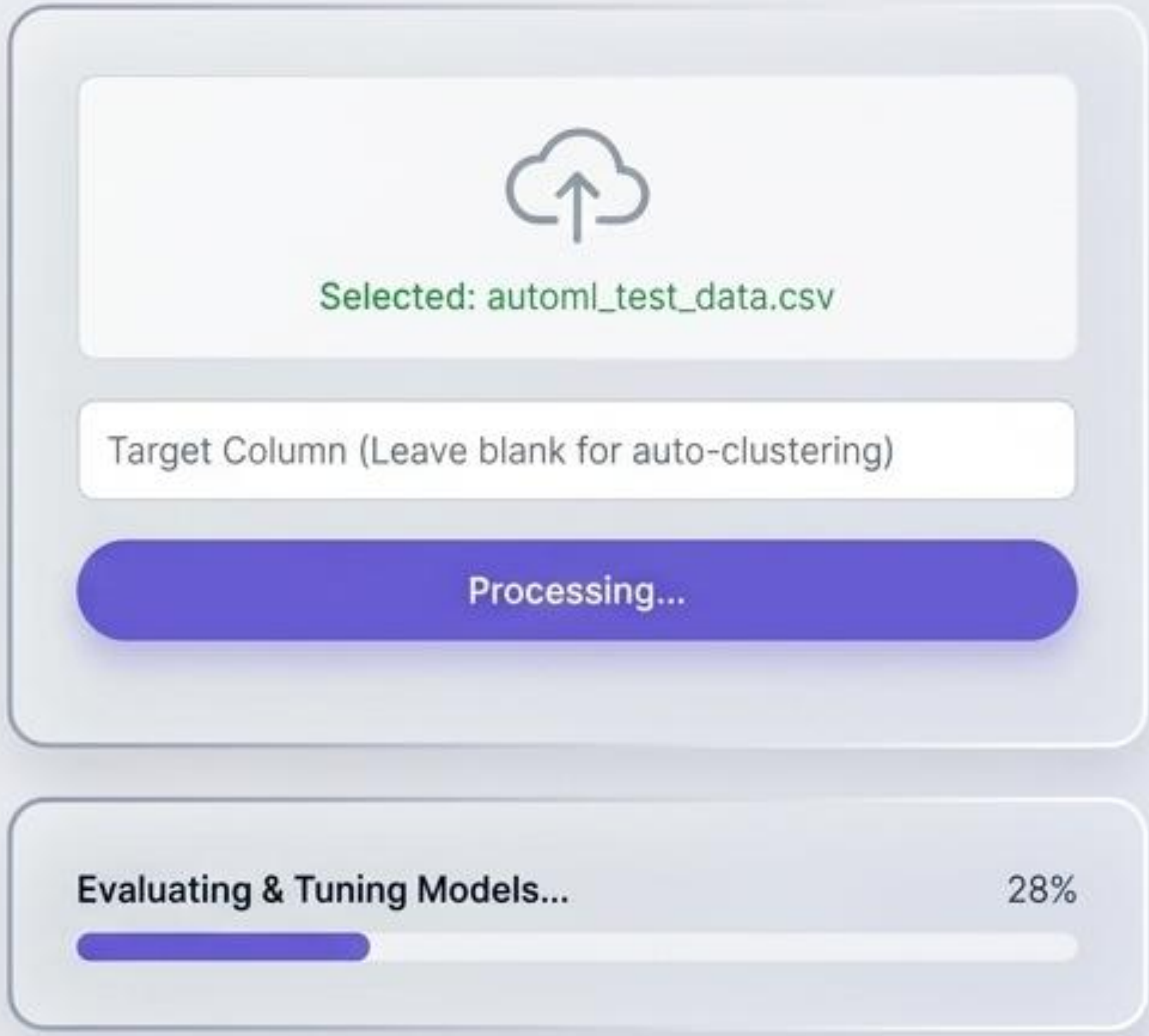
Zero server-blocking

# High-Level Architecture: The Microservice Blueprint



# Demystifying the Asynchronous Engine



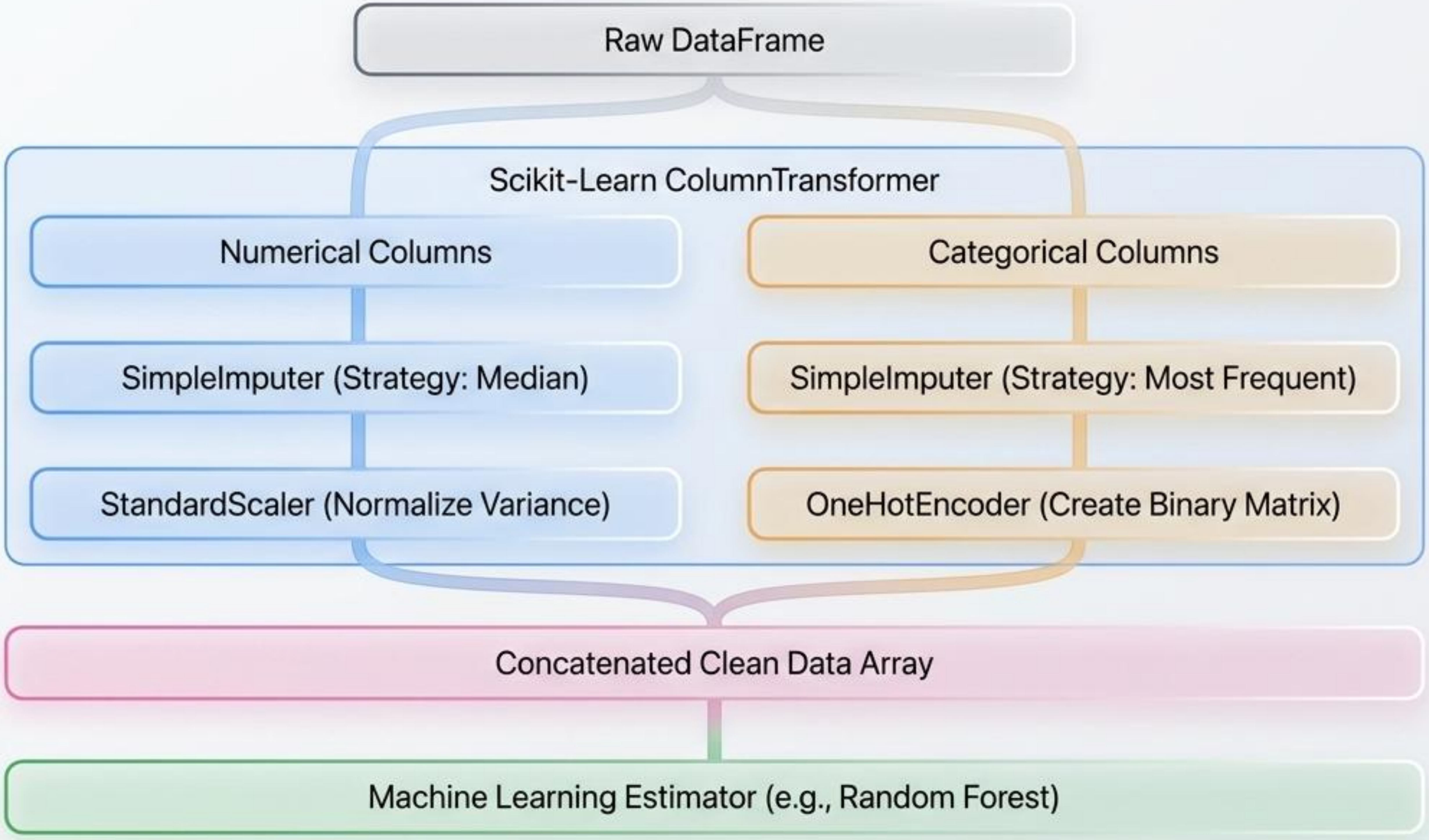

1. Next.js POSTs CSV

2. Flask saves file, hands ticket to Redis, returns 202 Accepted

3. Celery consumes task, begins heavy CPU training

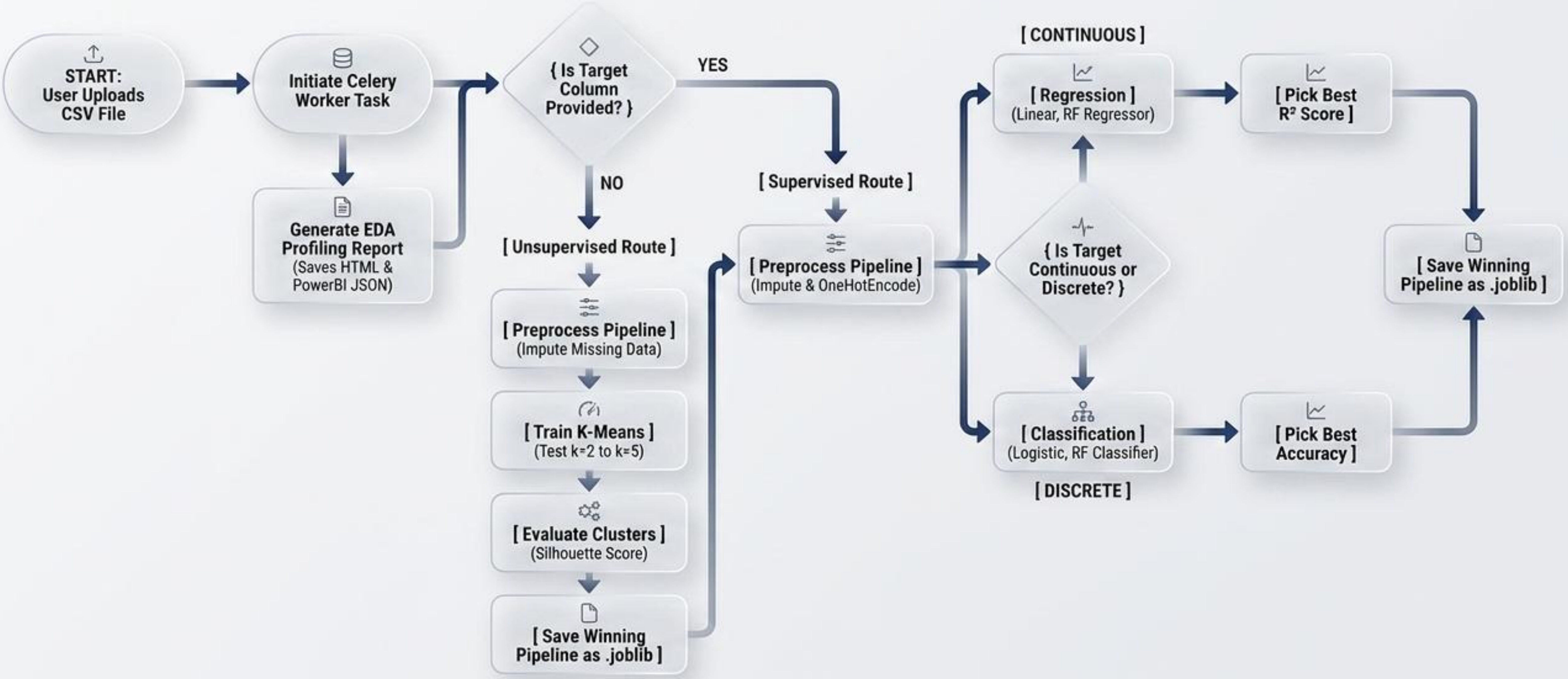
4. Next.js polls Flask for status without freezing browser

# Dynamic Preprocessing: Sanitizing the Data



# The Intelligent Routing Engine

Autonomous decision-making mimicking a data scientist.



# The Algorithmic Routing Matrix

Mathematical evaluation criteria for the routing engine.

## Regression
(Continuous Target)

**Algorithms Tested:**
- Linear Regression
- Random Forest Regressor

**Winning Metric:**
R-Squared ($R^2$) Score

## Classification
(Discrete Target)

**Algorithms Tested:**
- Logistic Regression
- Random Forest Classifier

**Winning Metric:**
Accuracy / F1-Score

## Clustering
(No Target Provided)

**Algorithms Tested:**
- K-Means (Iterating k=2 to k=5)

**Winning Metric:**
Silhouette Score

# Automated Exploratory Data Analysis (EDA)

Concurrent generation of rich statistical analytics.

## Real-Time Processing

Concurrent with model training, a background worker computes Pearson correlations, missing value matrices, and distribution geometries.

## Dual Export Architecture

1. **Interactive HTML**: Embedded directly into the Next.js UI.
2. **JSON Payload**: Exported for seamless ingestion into enterprise Business Intelligence tools like Power BI.

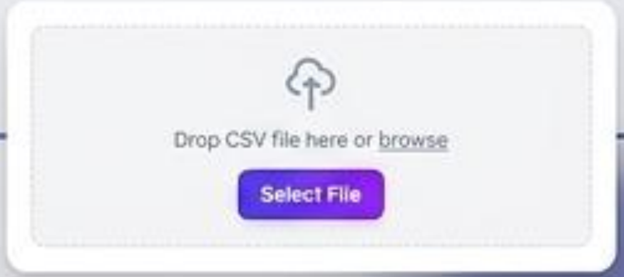
# React State Machine & User Experience

Managing the asynchronous polling lifecycle.

1

**State 1: Idle**

A polished Tailwind CSS drag-and-drop zone awaiting raw CSV ingestion.




2

**State 2: Polling**

The UI shifts to a **progress bar**, passively querying the Flask API while Celery processes the heavy ML workload.


3

**State 3: Completed**

The dashboard populates with dynamic **Metric Cards** displaying the optimal algorithm and evaluation score.

# Proof of Engine: Real-World Testing

Synthesizing dynamic routing across heterogeneous data.

### Real Estate Dataset

| Target: | System Action: | Winner: |
| --- | --- | --- |
| Property_Price (Continuous) | Triggered Regression Pipeline | Random Forest Regressor ($R^2 = 0.89$) |

### Telecom Churn Dataset

| Target: | System Action: | Winner: |
| --- | --- | --- |
| Churn_Status (Discrete) | Triggered Classification Pipeline | Random Forest Classifier (Accuracy = 86.2%) |

### Retail Customer Dataset

| Target: | System Action: | Winner: |
| --- | --- | --- |
| None Provided | Triggered Unsupervised Clustering | K-Means (k=3, Silhouette = 0.68) |

# The Serialized Pipeline Artifact

Frictionless deployment into production environments.

1. Fitted Imputers (Medians/Modes)

2. Fitted Encoders & Scalers


3. Winning ML Estimator

### The Naive Approach

Saving just the final model will crash in production when it receives unscaled, unencoded raw data.

### The Robust Solution

Our platform serializes the entire preprocessing pipeline alongside the algorithm into a single artifact, allowing instant inference on raw data.

# Architectural Future Scope

Evolving the foundation for enterprise scale.

### Big Data Integration

Extending beyond Pandas memory limits with Apache Spark for out-of-core, distributed computing.

### Deep Learning

Integrating PyTorch and TensorFlow to intelligently route image and sequential data pipelines.

### Unstructured NLP

Upgrading the preprocessor to handle complex text columns using Word2Vec or BERT embeddings prior to routing.

# Democratizing machine learning through scalable, asynchronous software engineering.

Thank you. Open for questions.

Next.js

Python

Docker

Redis

# Thank You!

**Any Questions?**

Project By: Jamil Aktar,  Jayed Islam, Dilruba Khatun

Domkal Institute of Engineering and Technology